



Sorbonne Université

Master 2 Science et Technologie du Logiciel

Développement des Algorithmes d'Application Réticulaire

**Projet final - Moteur de recherche d'une
bibliothèque**

Auteurs : Florian CODEBECQ, Nandraina RAZAFINDRAIBE

Responsable : Binh-Minh BUI-XUAN

Année scolaire : 2024/2025

Table des matières

1	Introduction	1
2	Définition de problème et structures de données utilisées	1
2.1	Définition du Problème de Recherche d'Information	1
2.2	Choix des technologies pour le moteur de recherche	1
2.3	Stockage et Indexation des Documents	1
2.4	Graphes de Jaccard et similarité documentaire	2
2.5	Indices de centralité et classement des résultats	2
2.6	Génération des suggestions	3
3	Présentation théorique des algorithmes	3
3.1	Algorithme de Knuth-Morris-Pratt (KMP)	3
3.2	Algorithme d'Aho-Ullman	3
3.3	Mesure de Similarité et graphe de Jaccard	4
3.4	Closeness Centrality	6
3.5	Betweenness Centrality	6
3.6	PageRank Centrality	7
4	Partie test : obtention des testbeds	7
4.1	Jeux de données utilisés	7
4.2	Jeux de données utilisés	8
4.3	Prétraitement des données	8
5	Tests de performance et résultats	8
5.1	Méthodologie de test	8
5.2	Temps d'exécution des algorithmes	8
6	Discussion des résultats	10
7	Conclusion	11

1 Introduction

Dans un monde où l'accès à l'information est primordial, les moteurs de recherche sont devenus des outils indispensables pour naviguer efficacement à travers les vastes collections de documents numériques. Ce projet s'inscrit dans cette problématique en proposant la conception et l'implémentation d'une application web et mobile dédiée à la recherche de documents textuels au sein d'une bibliothèque numérique.

Notre travail répond aux exigences du projet, qui vise à développer une application web et mobile permettant une recherche efficace de livres dans une bibliothèque numérique. Grâce à une base de données contenant plusieurs milliers d'ouvrages sous format textuel, l'objectif est de concevoir un moteur de recherche performant, capable de répondre aux requêtes des utilisateurs de manière pertinente et rapide.

L'approche adoptée repose sur l'exploitation d'algorithmes avancés pour l'indexation et le classement des résultats, en s'appuyant sur des concepts de centralité et de similarité. L'application ne se limite pas à une simple recherche par mots-clés, mais propose également des fonctionnalités avancées permettant une exploration plus fine du contenu textuel.

Au-delà de la conception technique, ce projet vise également à analyser la pertinence et la performance du moteur de recherche, en évaluant son comportement sur un grand volume de données. L'expérience utilisateur reste au cœur des préoccupations, avec une interface intuitive et une capacité à s'adapter aux besoins des lecteurs.

2 Définition de problème et structures de données utilisées

2.1 Définition du Problème de Recherche d'Information

Rechercher un livre dans une vaste collection nécessite un système capable d'extraire les documents pertinents en fonction de mots-clés spécifiques. Il s'agit de permettre à un utilisateur de retrouver des livres en fonction des mots ou expressions régulières qu'ils contiennent, dans une collection volumineuse où la recherche manuelle serait impraticable. Au-delà de la simple recherche textuelle, le système classe les documents selon leur pertinence et propose des suggestions basées sur la similarité entre les livres. Cette approche assure une navigation fluide et une exploration efficace du contenu disponible.

2.2 Choix des technologies pour le moteur de recherche

Le choix des technologies utilisées dans la création d'un moteur de recherche est primordial pour garantir non seulement des performances optimales, mais aussi une évolutivité à long terme. Dans le cadre de ce projet, Flask a été sélectionné pour le backend, tandis qu'Angular a été choisi pour le frontend, deux frameworks qui offrent une combinaison puissante pour répondre aux besoins du projet.

Flask, un framework léger et flexible basé sur Python, permet de construire rapidement des API performantes tout en facilitant l'intégration avec des bibliothèques Python puissantes. Python dispose d'un large écosystème d'outils dédiés à la manipulation de données, ce qui est particulièrement utile pour le traitement de texte et l'analyse de données dans le cadre d'un moteur de recherche.

Angular, quant à lui, est un framework JavaScript très puissant pour créer des applications web dynamiques et réactives. L'un de ses grands avantages est sa capacité à produire des applications multiplates-formes, c'est-à-dire utilisables non seulement sur des ordinateurs via des navigateurs web, mais aussi sur des smartphones et tablettes. Cela est rendu possible grâce à Angular qui permet de créer des applications responsive, adaptées à différents types de dispositifs avec une seule base de code. Ainsi, l'expérience utilisateur est fluide, peu importe l'appareil utilisé. Angular gère efficacement les interactions utilisateur en temps réel, assurant une recherche rapide et dynamique.

2.3 Stockage et Indexation des Documents

Pour répondre efficacement à ces exigences, nous avons conçu une architecture de stockage à deux niveaux. Les documents textuels, téléchargés depuis **The Gutenberg Project**[1], sont conservés dans leur format original (*.txt*) au sein d'un répertoire dédié, assurant ainsi leur intégrité. Leurs métadonnées sont parallèlement stockées au format JSON pour en faciliter l'exploitation et la représentation.

Afin d'optimiser les performances de recherche, nous avons implémenté en Python une table d'indexation sous forme de dictionnaire de données (*HashMap*). Chaque terme significatif y est associé à la liste des documents dans lesquels il apparaît, avec un score pondéré par TF-IDF. Cette structure clé permet d'accélérer les requêtes en évitant un parcours exhaustif des textes.

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

$$\text{TF}(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$$

$$\text{IDF}(t) = \log \left(\frac{N}{|\{d \in D : t \in d\}|} \right)$$

où :

- $\text{TF}(t, d)$ (**Term Frequency**) : fréquence du terme t dans le document d , normalisée par le nombre total de termes dans d .
- $\text{IDF}(t)$ (**Inverse Document Frequency**) : mesure l'importance du terme t en fonction de son apparition dans l'ensemble des documents D .
- $f_{t,d}$: fréquence brute du terme t dans le document d .
- N : nombre total de documents.
- $|\{d \in D : t \in d\}|$: nombre de documents contenant le terme t .

Pour faciliter l'obtention des scores TF-IDF, Python offre la méthode `TfidfVectorizer` à travers sa bibliothèque `scikit-learn`[2]. La complexité globale de cette méthode est estimée comme $O(n*m)$ sur un corpus de n documents contenant un total de m mots avec des optimisations internes utilisées par `scikit-learn`.

Cette méthode est exécutée sur l'ensemble des livres dupliqués temporairement. Cela nous permet de répéter une quinzaine de fois le nom de l'auteur et une dizaine de fois le titre sans modifier les livres originaux. Ainsi, les titres et les auteurs seront mis en évidence grâce à leurs multiples occurrences dans les livres. Une fois le traitement terminé, nous supprimerons ces documents temporaires afin de libérer de l'espace.

Quant à la construction de la table d'index, nous avons pris en entrée le résultat de la méthode `TfidfVectorizer` (une *matrice d'adjacence*). Ensuite, en $O(T \times N)$ où T étant le nombre de termes et N étant le nombre de livres, nous obtenons la table d'index dans laquelle chaque terme est associé aux livres qui le concernent[3] pour construire notre liste d'adjacence.

2.4 Graphes de Jaccard et similarité documentaire

Pour mettre en œuvre les fonctionnalités de suggestion, nous avons construit un graphe de Jaccard représenté par une matrice d'adjacence. Chaque livre est représenté comme un nœud dans un graphe. Cette structure modélise les relations de similarité entre les documents, chaque arête étant pondérée selon la distance de Jaccard calculée entre deux livres. La liaison entre 2 nœuds est établie si la distance entre ces 2 nœuds est inférieure à un seuil (0.5 dans notre cas). Cette mesure, basée sur le rapport entre l'intersection et l'union des ensembles de termes contenus dans les documents, permet d'identifier des textes thématiquement proches même lorsqu'ils ne partagent pas exactement les mêmes mots-clés recherchés par l'utilisateur.

La computation d'un tel algorithme s'exécute en $O(N^2 \times (k_1 + k_2))$ où le nombre total de paires de livres est $\frac{N \times (N-1)}{2}$, soit $O(N^2)$. k_1 et k_2 sont respectivement les tailles des ensembles de mots-clés pour les livres pris deux à deux. Ce modèle permet non seulement d'améliorer la classification des résultats de recherche, mais aussi d'offrir des recommandations pertinentes aux utilisateurs.

2.5 Indices de centralité et classement des résultats

Le classement pertinent des résultats constitue un aspect essentiel de notre moteur de recherche. Pour dépasser le simple comptage d'occurrences, nous avons implémenté des indices de centralité attribués à chaque document dans le graphe de Jaccard. Ces indices, calculés selon des algorithmes de théorie des graphes, permettent d'évaluer l'importance relative de chaque livre au sein de l'ensemble de la bibliothèque. Un document possédant une forte centralité sera considéré comme plus représentatif ou influent, et se verra donc attribuer un rang supérieur dans les résultats de recherche.

Pour la pondération des résultats, on a utilisé cette règle suivante :

Pour chaque document d_i dans les résultats, le score de pertinence est calculé comme suit

$$P(d_i) = tfidf_i \times (c_i + \epsilon)$$

Où :

- $tfidf_i$ est le score TF-IDF du document d_i .
- c_i est le score de centralité du document d_i .
- ϵ est un petit biais ajouté pour éviter que les documents sans score de centralité aient un score nul.

Les résultats sont ensuite triés par ordre décroissant de leur score de pertinence. Les indices de centralité, tels que le betweenness, le closeness et le PageRank, sont déjà implémentés dans la bibliothèque *NetworkX*[4] de Python. La matrice d’adjacence, la table d’index ainsi que les indices de centralité sont stockés dans un objet sérialisé Python afin de faciliter l’accès aux données lors de la recherche.

L’utilisateur pourra spécifier l’indice de centralité de son choix pour le classement dans la recherche avancée. Nous détaillerons davantage ces mesures de centralité dans la section suivante.

2.6 Génération des suggestions

Dans le cadre de notre moteur de recherche de livres, il est essentiel d’offrir aux utilisateurs des recommandations pertinentes basées sur leurs recherches initiales. Pour ce faire, nous avons mis en place un algorithme de génération de suggestions s’appuyant sur une structure de graphe représentée par une matrice d’adjacence.

L’algorithme repose sur l’exploitation des relations entre les livres afin d’identifier des suggestions pertinentes. Nous partons des meilleurs résultats retournés par la recherche, qui correspondent aux documents les plus pertinents selon la requête de l’utilisateur. Ensuite, nous explorons la matrice d’adjacence pour extraire les documents voisins de ces résultats initiaux. Les résultats les plus pertinents constituent les sommets (livres) privilégiés en premier.

Le processus s’arrête dès qu’un nombre maximal de suggestions est atteint, garantissant ainsi des recommandations pertinentes et limitées en quantité pour optimiser l’expérience utilisateur.

3 Présentation théorique des algorithmes

3.1 Algorithme de Knuth-Morris-Pratt (KMP)

L’algorithme de Knuth-Morris-Pratt (KMP) est utilisé pour rechercher un motif dans un texte en temps linéaire. Contrairement à l’approche naïve, KMP évite les comparaisons redondantes en utilisant les informations des correspondances partielles déjà établies.

Fonctionnement :

- *Prétraitement* : KMP commence par prétraiter le motif F pour générer un tableau de préfixes qui est aussi de suffixe ou LPS (Longest Proper Prefix). Cette table contient l’information sur les préfixes et suffixes qui se répètent dans le motif. La longueur de cette table est la même que celle du motif. On peut obtenir le LPS en utilisant cet algorithme :

Pour tout i , nous avons :

$$\text{LPS}[i] = \text{taille du plus long préfixe-suffixe de } F[0 \dots i - 1] \quad (1)$$

$$\text{Si } F[i] = F[\text{LPS}[i]] \text{ alors } \text{LPS}[i] = \text{LPS}[\text{LPS}[i]] \quad (2)$$

(2) correspond à une optimisation de LPS.

- *Recherche* : Lors de la recherche dans le texte, si une erreur de correspondance se produit, l’algorithme utilise la table pour sauter les comparaisons inutiles, ce qui permet d’éviter de revenir en arrière dans le texte.

Comparaison avec d’autres algorithmes :

- *Naïve* : L’algorithme naïf compare chaque caractère du motif avec chaque caractère du texte, ce qui donne une complexité de $O(T \times M)$. L’algorithme KMP améliore cette complexité en $O(T + M)$.
- *Boyer-Moore* : Le Boyer-Moore [5] est généralement plus rapide que KMP, surtout pour les textes longs, en exploitant des heuristiques de saut basées sur les caractères du motif. Cependant, KMP peut être plus rapide sur des motifs courts ou des textes avec peu de répétitions.

3.2 Algorithme d’Aho-Ullman

L’algorithme Aho-Ullman est une méthode efficace pour effectuer une recherche simultanée de plusieurs motifs dans un texte. Cette approche consiste à construire un automate fini à partir d’une expression régulière. Cet algorithme d’Aho-Ullman [6, Chapter 10] comporte plusieurs étapes :

- *Construction d’un arbre syntaxique à partir d’une expression régulière*. Cette étape consiste à transformer l’expression régulière en un arbre syntaxique qui représente sa structure logique. Cet arbre décompose l’expression en opérateurs (concaténation, union, fermeture de Kleene) et opérandes (symboles de l’alphabet).

- *Conversion de cet arbre en un automate fini non déterministe (AFND) avec ϵ -transition.* L'automate fini non déterministe avec ϵ -transitions (AFND- ϵ) est construit en appliquant les algorithmes de Thompson : Chaque feuille de l'arbre syntaxique est un état avec une transition associée. Les nœuds internes appliquent les règles de concaténation, union et étoile de Kleene[7]. Des transitions ϵ sont utilisées pour relier les états intermédiaires, facilitant ainsi la construction.

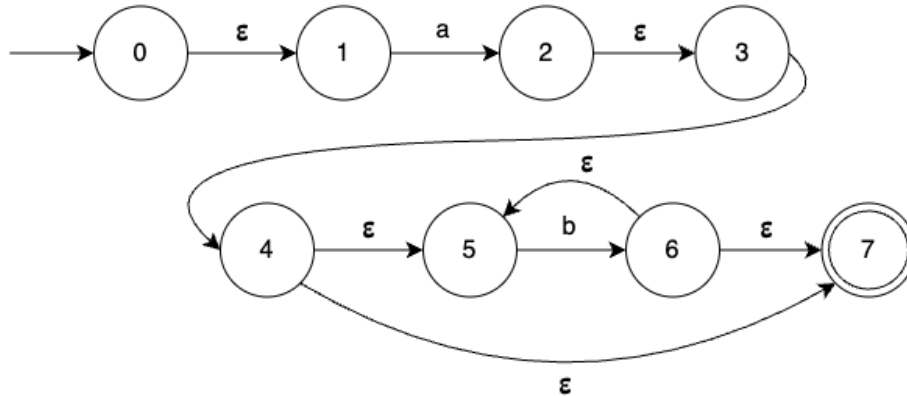


FIGURE 1 – Représentation AFND de $a.b^*$
[8]

- *Transformer de l'automate fini non déterministe en un automate fini déterministe (AFD).* L'automate fini déterministe (AFD) est obtenu en éliminant les transitions ϵ via l'algorithme de déterminisation par construction des sous-ensembles (Subset Construction) :
 - Chaque état de l'AFD correspond à un ensemble d'états de l'AFND.
 - On fusionne les transitions possibles pour chaque symbole afin de garantir l'unicité du prochain état.

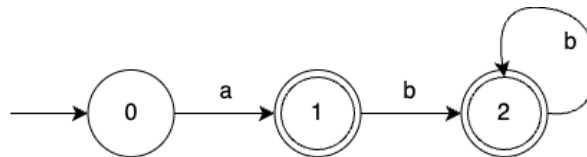


FIGURE 2 – Représentation AFD de $a.b^*$
[8]

- *Optimiser l'AFD afin d'obtenir un automate avec un nombre minimum d'états.* L'AFD peut être réduit pour minimiser le nombre d'états, en appliquant l'algorithme de minimisation de Hopcroft :
 - On regroupe les états équivalents en classes d'équivalence.
 - On élimine les états inutiles et fusionne ceux ayant le même comportement.
 L'automate résultant est minimal, garantissant un coût de reconnaissance optimal.

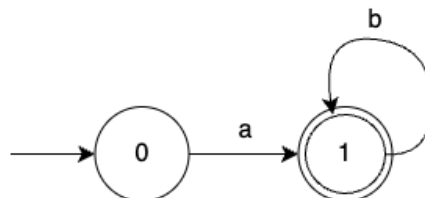


FIGURE 3 – Représentation AFD optimisée de $a.b^*$
[8]

3.3 Mesure de Similarité et graphe de Jaccard

La mesure de similarité de Jaccard est utilisée pour comparer deux ensembles et déterminer leur degré de ressemblance. Elle est particulièrement utilisée dans l'analyse de texte, la classification de documents, et la détection

de similarité entre corpus textuels ou ensembles de données.

Soient A et B deux ensembles représentant respectivement les termes présents dans les documents D_A et D_B .

$J(A, B)$ est la similarité de Jaccard, donnée par la formule :

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

La distance de Jaccard est donc :

$$d_J(A, B) = 1 - J(A, B)$$

Cette distance varie entre 0 et 1. Lorsque $d_J(A, B) = 0$, les ensembles sont identiques. À l'inverse, si $d_J(A, B) = 1$, cela signifie que les ensembles sont totalement différents et n'ont aucune intersection.

Le graphe de Jaccard est obtenu en calculant la distance de Jaccard entre les livres pris deux à deux. Les résultats forment ensuite la matrice d'adjacence de Jaccard, permettant ainsi de représenter le graphe. La liaison entre les nœuds est établie si les deux sont assez similaires. Dans le projet, nous avons fixé le seuil de similarité à 0.5. Ce seuil a été choisi de manière à ce que seules les paires de livres présentant une similarité modérée ou élevée soient connectées. En effet, une distance de Jaccard de 0 indique que les deux livres sont identiques, tandis qu'une distance de 1 signifie qu'ils ne partagent aucun terme en commun. En fixant le seuil à 0.5, nous garantissons que les livres ayant une similarité plus significative sont reliés, ce qui permet de mieux représenter les relations entre les livres dans le graphe de Jaccard (voir Figure 4). Cela permet d'éviter une trop grande dispersion des liens et de maintenir une pertinence dans les connexions formées. Avec ce seuil, nous obtenons 123 connexions, soit 123 arêtes, chacune pondérée par la distance de Jaccard inférieure à 0.5.

Graph des livres avec leurs distances Jaccard

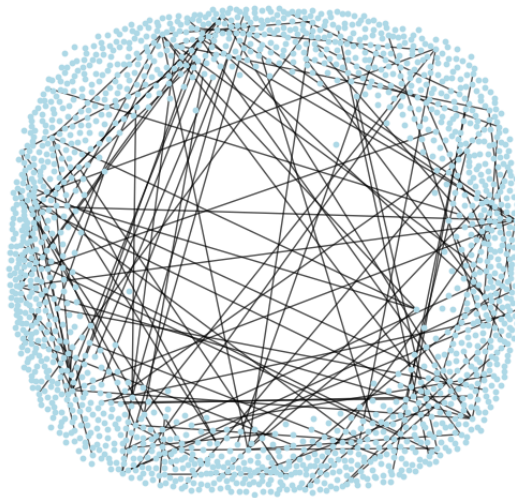


FIGURE 4 – Graphe de Jaccard avec 123 arêtes
[9]

Il en existe d'autres mesures de similarité comme *Cosinus* et *Dice*. La similarité cosinus est utilisée pour mesurer la proximité angulaire entre deux vecteurs représentant des documents. Contrairement à Jaccard, elle prend en compte les fréquences des termes. Elle est adaptée aux documents longs avec pondération TF-IDF mais ne distingue pas les mots rares des mots fréquents.

Soient A et B deux vecteurs représentant des documents dans un espace de dimension n , où chaque dimension correspond à un terme du vocabulaire. Les composantes de ces vecteurs sont les scores TF-IDF des termes présents dans les documents.

La similarité cosinus entre ces deux documents est donnée par :

$$\cos(A, B) = \frac{\sum_{i=1}^n A_i B_i}{\sqrt{\sum_{i=1}^n A_i^2} \times \sqrt{\sum_{i=1}^n B_i^2}}$$

où $A = (A_1, A_2, \dots, A_n)$ et $B = (B_1, B_2, \dots, B_n)$ sont les vecteurs TF-IDF des documents.

La similarité de Dice est une variante de Jaccard avec un poids supplémentaire sur les éléments communs. Dice est plus sensible aux petits ensembles mais moins interprétable que Jaccard sur de grands corpus.

Soient A et B deux ensembles représentant respectivement les termes présents dans les documents D_A et D_B .

La similarité de Dice entre ces deux documents est donnée par :

$$D(A, B) = \frac{2|A \cap B|}{|A| + |B|}$$

Lee (1999)[10] dans "Measures of distributional similarity" a démontré que "la similarité cosinus offre généralement de meilleurs résultats pour les comparaisons de textes complets, tandis que Jaccard est plus adapté pour les représentations par ensembles de mots-clés ou de caractéristiques discrètes."

3.4 Closeness Centrality

La centralité de proximité (*Closeness Centrality*) est une mesure qui quantifie la proximité d'un nœud par rapport à tous les autres nœuds dans un graphe. Dans le contexte de notre bibliothèque numérique, elle permet d'identifier les documents qui sont "proches" de nombreux autres documents en termes de contenu, suggérant ainsi leur caractère représentatif ou central dans la collection.

$$C_C(v) = \frac{n - 1}{\sum_{u \neq v} d(v, u)}$$

où :

- n est le nombre total de nœuds dans le graphe,
- $d(v, u)$ est la distance la plus courte entre le nœud v et un autre nœud u .

Plus $C_C(v)$ est grand, plus le nœud v est "central" dans le graphe, car il est proche des autres nœuds en termes de distance. Comme l'expliquent Wasserman et Faust dans "Social Network Analysis : Methods and Applications" (1994)[11], les nœuds avec une forte centralité de proximité peuvent diffuser efficacement l'information à travers le réseau. Cela peut être utile dans des systèmes de recommandation de bibliothèques, où l'on cherche à mettre en avant les documents les plus "reliés" à d'autres.

3.5 Betweenness Centrality

La centralité d'intermédierité (*Betweenness Centrality*) mesure la fréquence à laquelle un nœud se trouve sur les plus courts chemins entre d'autres nœuds du graphe. Un nœud avec une centralité de betweenness élevée agit comme un "pont" entre différents sous-graphes et joue un rôle crucial dans la connectivité globale du réseau.

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

où :

- $\sigma_{st}(v)$ est le nombre de ces chemins qui passent par le nœud v .
- σ_{st} est le nombre total de plus courts chemins entre les nœuds s et t ,

Freeman (1977)[12], dans son article fondateur "A Set of Measures of Centrality Based on Betweenness", explique que "les nœuds à forte intermédierité contrôlent le flux d'information entre différentes parties du réseau." Cette propriété est particulièrement utile dans notre moteur de recherche pour identifier les ouvrages interdisciplinaires ou ceux qui établissent des liens entre des domaines distincts.

3.6 PageRank Centrality

Développé initialement par Brin et Page pour le moteur de recherche Google[13], *PageRank* est un algorithme qui attribue une pondération numérique à chaque élément d'un ensemble de documents liés par des hyperliens, dans le but de mesurer son importance relative au sein de cet ensemble. Dans notre adaptation, nous considérons les liens de similarité entre documents comme des "votes" d'importance, similaires aux hyperliens sur le web.

Le score PageRank $PR(A)$ d'un document A est calculé de manière itérative par :

$$PR(A) = (1 - d) + d \times \sum_{i \in In(A)} \frac{PR(i)}{L(i)}$$

où $PR(A)$ est le PageRank de la page A , $In(A)$ est l'ensemble des pages pointant vers A , $L(i)$ est le nombre de liens sortants de la page i , et d est un facteur d'amortissement (souvent pris égal à 0.85).

Il existe aussi un autre indice, *HITS* (*Hyperlink-Induced Topic Search*), qui constitue une approche complémentaire. Contrairement aux autres mesures qui attribuent un score unique de centralité, *HITS* distingue deux rôles distincts au sein du réseau : Le score d'autorité, qui évalue la pertinence d'un document en fonction du nombre et de la qualité des liens pointant vers lui. Le score de hub, qui mesure la capacité d'un document à orienter vers des ressources pertinentes. Cette distinction permet d'identifier à la fois les documents de référence riches en contenu (autorités) et ceux qui jouent un rôle d'aiguillage vers des informations importantes (hubs). Toutefois, cette sophistication implique une complexité de calcul plus élevée et une stabilité moindre par rapport aux approches comme *PageRank*.

On peut voir dans le tableau ci-dessous (voir Table 1) un extrait des indices de centralité de notre graphe d'adjacence. Pour rappel, les nœuds représentent les livres.

Node	Betweenness	Closeness	PageRank
766.txt	0.000136	0.029896	0.007640
917.txt	0.000227	0.032886	0.007460
968.txt	0.000136	0.032886	0.007348
52521.txt	0.000045	0.021477	0.006839
996.txt	0.000091	0.013423	0.009737
141.txt	0.000091	0.013423	0.009737
2591.txt	0.000181	0.026846	0.009276
821.txt	0.000816	0.041107	0.010865
967.txt	0.000227	0.032886	0.007497
19068.txt	0.000045	0.021477	0.007204
20.txt	0.000091	0.013423	0.009737

TABLE 1 – Extrait des résultats des indices de centralité pour 12 livres.
[14]

Le livre 821.txt se distingue comme étant le plus central dans le graphe, avec des valeurs élevées dans les trois indices de centralité : betweenness, closeness et PageRank. Cela suggère qu'il joue un rôle clé dans le réseau, agissant comme un médiateur entre d'autres livres, pouvant atteindre rapidement tous les autres livres et étant influencé par des livres importants. En revanche, les livres avec des indices plus faibles, comme 52521.txt ou 19068.txt, sont moins centraux et influents dans le graphe.

4 Partie test : obtention des testbeds

4.1 Jeux de données utilisés

Nous avons utilisé un jeu de données constitué de 1664 livres provenant de Gutenberg.org, une bibliothèque numérique contenant des livres du domaine public. Ces livres sont disponibles dans des fichiers texte brut, ce qui nous permet de les analyser directement. Nous avons spécifiquement sélectionné uniquement des livres en anglais, ce qui a facilité le traitement, notamment en évitant la gestion de caractères spéciaux dans d'autres langues. De plus, nous avons choisi uniquement les livres contenant plus de 10 000 caractères afin d'assurer une taille suffisante pour une analyse pertinente.

4.2 Jeux de données utilisés

Les fichiers utilisés proviennent du site Project Gutenberg [1], qui propose une vaste collection de livres gratuits et accessibles. Nous avons extrait ces livres via l'API de Gutendex (<https://www.gutendex.org/>), qui, en plus du texte brut, nous fournit les métadonnées associées à chaque livre (ID, auteur, titre, langue, image, etc.). Un script Python a été développé pour automatiser le téléchargement des livres en interrogeant l'API. Chaque fichier est au format texte brut, ce qui simplifie grandement leur traitement.

4.3 Prétraitement des données

Avant d'utiliser les textes pour l'analyse, nous avons appliqué plusieurs étapes de prétraitement :

- *Tokenisation* : Nous avons utilisé la bibliothèque *NLTK* pour tokeniser les textes et obtenir une liste de mots à partir de chaque livre. Cette bibliothèque est notamment utilisée lors de la construction du graphe de Jaccard. Lors de la construction de la table d'index, la méthode *TfidfVectorizer* intègre déjà un tokeniseur de texte.
- *Nettoyage* : Le texte brut a été nettoyé en supprimant les caractères spéciaux, en convertissant tous les mots en minuscules et en éliminant les mots de moins de 3 caractères. Les mots ainsi retenus sont stockés dans une liste. Dans le calcul de la similarité de Jaccard, ces mots sont placés dans un ensemble (*set en Python*).
- *Suppression des stopwords* : Nous avons utilisé la liste de stopwords fournie par la bibliothèque python *NLTK* pour éliminer les mots courants qui n'apportent pas d'information pertinente. Cela permet de se concentrer sur les mots significatifs dans chaque texte.

5 Tests de performance et résultats

5.1 Méthodologie de test

Pour réaliser les tests, nous allons examiner une à une les différentes méthodes de recherche disponibles dans l'application, à savoir : la recherche par mot-clé, la recherche par expression régulière (regex) et la recherche avancée, qui permet de classer les résultats par ordre de pertinence tout en proposant des suggestions appropriées. Pour chaque test, nous utiliserons cinq mots-clés ou expressions régulières et analyserons les résultats obtenus en fonction du temps d'exécution ainsi que du nombre de livres retournés.

5.2 Temps d'exécution des algorithmes

- *Recherche simple* : Le test réalisé avec la recherche simple montre une bonne performance en termes de temps d'exécution. Dans une bibliothèque de 1 664 livres, la recherche avec 5 termes, que ce soit par mot-clé ou par une expression régulière (voir Figure 5 et Figure 6), n'a pas dépassé 0.28ms.

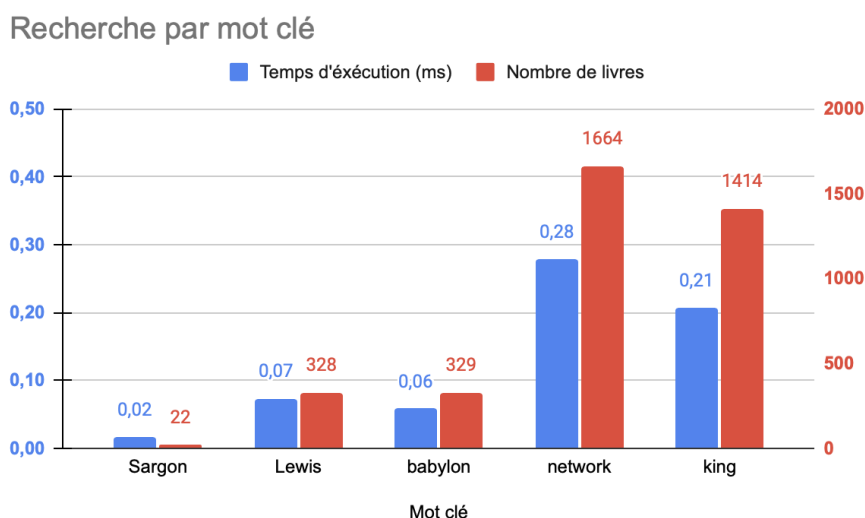


FIGURE 5 – Diagramme en bâtons de la recherche simple par mot clé
[15]

Recherche par expression régulière

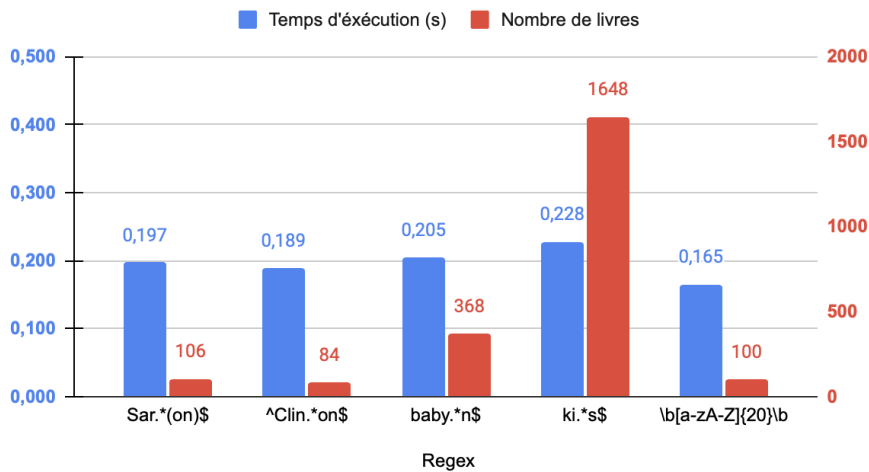


FIGURE 6 – Diagramme en bâtons de la recherche simple par expression régulière [15]

— *Recherche avancée* : On pourrait s'attendre à ce qu'une recherche par expression régulière soit lente en raison du parcours des différents motifs qu'elle contient. Cependant, nos tests montrent de bonnes performances, avec un temps d'exécution moyen de 0,30ms. Dans le pire des cas, la recherche a nécessité 7,03ms pour parcourir tous les livres et proposer des suggestions. Afin d'optimiser cela, nous avons décidé de limiter la liste des suggestions à 100 résultats (voir Figure 7 et Figure 8). L'algorithme est conçu pour s'arrêter dès que ce seuil est atteint, en s'appuyant sur les voisinages.

Ranking de recherche par mot clé

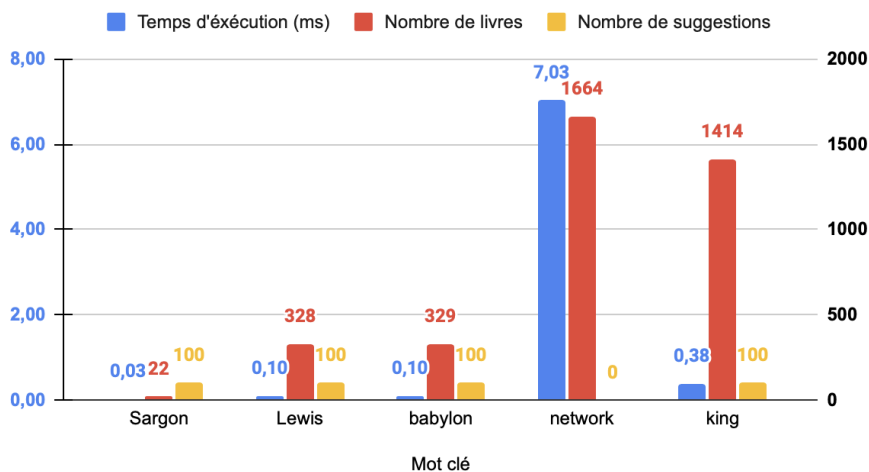


FIGURE 7 – Diagramme en bâtons de la recherche avancée par mot clé [15]

Ranking de recherche par expression régulière

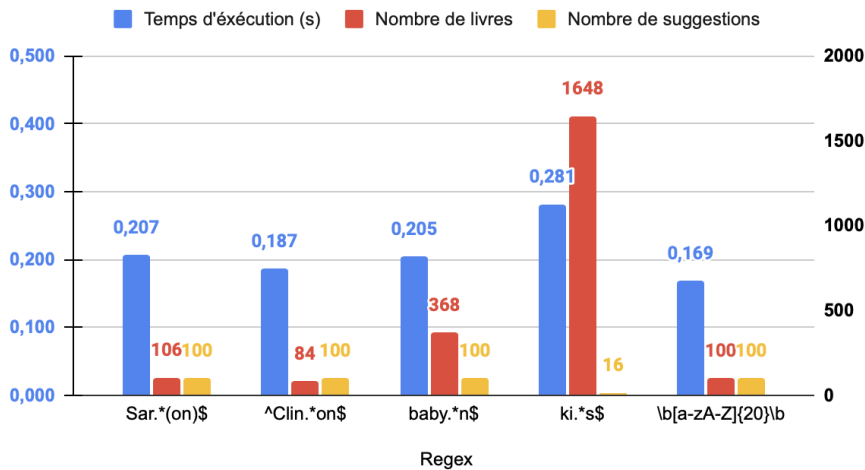


FIGURE 8 – Diagramme en bâtons de la recherche avancée par expression régulière [15]

6 Discussion des résultats

Avoir un moteur de recherche qui répond dans les meilleurs délais possibles est toujours l'idéal. Lors de nos tests de performance, nous avons observé que nos algorithmes de recherche retournent les résultats quasiment instantanément. Cette rapidité s'explique par le choix de stockage de la table d'index (Liste d'adjacence), qui est un objet sérialisé en Python (*.pkl*) sous forme de dictionnaire (*hashmap*). Cet objet permet d'obtenir en temps constant $O(1)$ les listes de livres associées à chaque terme recherché. De plus, d'autres objets, tels que la matrice d'adjacence et les indices de centralité, sont pré-calculés pour éviter de les recalculer à chaque recherche. Il suffit simplement de charger ces objets lors de chaque requête.

Cependant, bien que l'utilisation de fichiers sérialisés soit efficace pour de petites à moyennes tailles de données, elle présente certaines limitations, notamment pour des objets volumineux. La sérialisation d'objets volumineux peut entraîner des problèmes de mémoire et de performance, notamment lors du chargement ou de la sauvegarde de fichiers. En outre, les fichiers sérialisés peuvent devenir difficiles à gérer et à maintenir à mesure que la taille des données augmente.

En termes de sécurité, la sérialisation d'objets Python peut introduire des vulnérabilités, surtout lorsque des fichiers sérialisés sont chargés depuis des sources non fiables. Un attaquant pourrait potentiellement injecter du code malveillant dans un fichier sérialisé.

Pour les objets de grande taille, une alternative consiste à utiliser une base de données relationnelle optimisée pour le stockage et l'accès rapide aux données, comme *SQLite* pour des applications locales ou *PostgreSQL* pour des applications plus robustes et à grande échelle. Ces bases de données permettent de gérer des volumes de données plus importants de manière évolutive. Elles disposent de mécanismes d'indexation qui accélèrent considérablement les recherches en permettant une récupération rapide des données pertinentes sans avoir à parcourir l'intégralité des enregistrements.

En résumé, bien que la sérialisation d'objets Python offre une solution rapide et pratique pour des données de petite taille, il est judicieux d'opter pour des solutions plus adaptées à grande échelle, comme les bases de données ou les caches en mémoire, afin de garantir la performance à long terme.

7 Conclusion

Références

- [1] Project Gutenberg. Project gutenberg - free ebooks, 2024. Consulté le 12 mars 2025.
- [2] Scikit learn Developers. sklearn TfidfVectorizer, 2024., 2024. Consulté le 12 mars 2025.
- [3] Binh-Minh Bui-Xuan. Développement des algorithmes d'application réticulaire : Supports de cours, 2024. Consulté dans le cadre du projet.
- [4] NetworkX Developers. Networkx centrality algorithms, 2024. Consulté le 12 mars 2025.
- [5] Thierry Lecroq. A variation on the boyer-moore algorithm. *Theoretical Computer Science*, 92(1) :119–144, 1992.
- [6] Alfred V Aho and Jeffrey D Ullman. *Foundations of computer science*. Computer Science Press, Inc., 1992.
- [7] Stephen Cole Kleene. Representation of events in nerve nets and finite automata. *CE Shannon and J. McCarthy*, 1951.
- [8] Razafindraibe Nandraina. Automate, 2025. Réalisé dans le cadre du projet.
- [9] Razafindraibe Nandraina. Graphe de jaccard, networkx, 2025. Réalisé dans le cadre du projet.
- [10] Lillian Lee. Measures of distributional similarity. *arXiv preprint cs/0001012*, 2000.
- [11] Stanley Wasserman and Katherine Faust. Social network analysis : Methods and applications. 1994.
- [12] Linton C Freeman. A set of measures of centrality based on betweenness. *Sociometry*, pages 35–41, 1977.
- [13] Sergey Brin and Lawrence Page. The anatomy of a large-scale hypertextual web search engine. *Computer networks and ISDN systems*, 30(1-7) :107–117, 1998.
- [14] Razafindraibe Nandraina. Indice de centralité, 2025. Réalisé dans le cadre du projet.
- [15] Codebecq Florian. Graphique, excel, 2025. Réalisé dans le cadre du projet.